Lecture 23 - WCOJ

Goals

- Present some background on WCOJ from Skew Strikes Back: New Developments in the Theory of Join Algorithms
- Present FreeJoin paper extends it and makes it practical

Warmup - Yannakakis

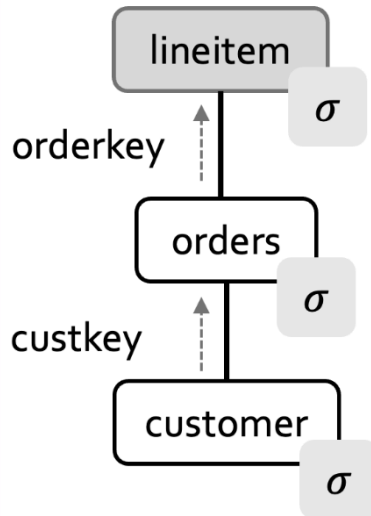
Query Hypergraph and Acyclic Property

- In the hypergraph of a query:
  - vertices are variables (attributes)
  - hyperedges are relations.
- Acyclicity means the "joining structure" of the query has no circular dependencies
  - Can't go from a vertex following hyperedges back to the same vertex
- Checking acyclicity:
  - **GYO algorithm** (Graham, Yu & Özsoyoglu 1979) decides acyclicity in PTIME via repeated removal:
    1. **Remove isolated attributes** (appear in only one relation)
    2. **Remove redundant relations**:
       - If relation $R_i \subseteq R_j$ (attribute-wise), remove R_i
    - If the hypergraph reduces to **empty**, the query is **acyclic**
    - Otherwise, it is cyclic
  - Constructive: also gives the **join tree**
  - *Example:*
    - Triangle query $R(x, y) \bowtie S(y, z) \bowtie T(x, z)$ is cyclic:
      - no attribs can be removed, and no relation is covered by another relation, so GYO is stuck
    - Clover query $R(x, a) \bowtie S(x, b) \bowtie T(x, c)$ is acyclic

Yannakakis's Algorithm

- Yannakakis (1981) showed that acyclic conjunctive queries (even with aggregation or projection) can be evaluated in time $O(|D| + |\text{output}|)$:
  - linear in the size of the input database plus the output.
  - essentially *instance-optimal*
- Crucially, **no dependence on intermediate result sizes**, which can be much larger than either the input or output for cyclic queries
- Proceeds in two passes over the join tree:
  1. **Semijoin reduction (bottom-up, then top-down)**
     - Push selections via semijoins along the join tree
     - Eliminates tuples that cannot participate in the final result
  2. **Final join (tree-structured)**
     - Perform joins in the join tree
- From paper I can't believe it's not Yannakakis, CIDR'26
  - Query

```sql
SELECT L_ORDERKEY,
       SUM(L_EXTENDEDPRICE*(1-L_DISCOUNT)),
       O_ORDERDATE,
       O_SHIPPRIORITY
FROM   CUSTOMER,
       ORDERS,
       LINEITEM
WHERE C_MKTSEGMENT = 'BUILDING' AND
      C_CUSTKEY = O_CUSTKEY AND
      L_ORDERKEY = O_ORDERKEY AND
      O_ORDERDATE < '1995-03-15' AND
      L_SHIPDATE > '1995-03-15'
GROUP BY L_ORDERKEY,
         O_ORDERDATE,
         O_SHIPPRIORITY
ORDER BY REVENUE DESC,
         O_ORDERDATE
```



- Yannakakis passes
  - Initial:

$$\text{// bottom-up:} \quad O^{\perp} \leftarrow O \ltimes_{\text{custkey}} C, \quad L^{\perp} \leftarrow L \ltimes_{\text{orderkey}} O^{\perp}$$

$$\text{// top-down:} \quad O^{\top} \leftarrow O^{\perp} \ltimes_{\text{orderkey}} L^{\perp}, \quad C^{\top} \leftarrow C \ltimes_{\text{custkey}} O^{\top}$$

  - Final: $C^T \bowtie O^T \bowtie L^T$
  - All tuples left in the final step are "useful", i.e., they contribute to output
  - Each semi-join is assumed to be implemented in $O(N)$, e.g., via hashing
    - can be improved further using Bloom filters

A Brief History of WCOJ

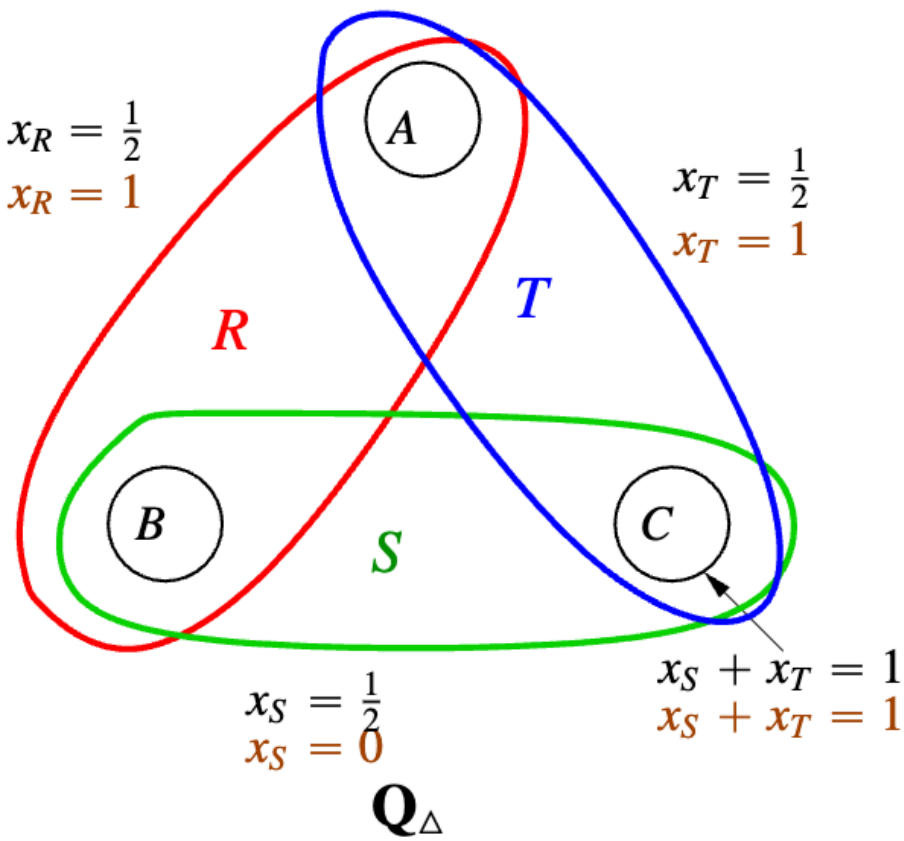
- Complexity notions in databases are defined in terms of data or query complexity, or combined complexity
  - typically query $\ll$ data
- For most of our discussion we will focus on *data complexity* for queries
- When reasoning about data complexity for joins, there is a dependence on:
  - structure, typically bounded "width" - acyclic is actually query width = 1
    - lots of other results in the theory literature going beyond acyclic
  - cardinalities
- AGM (Atserias Grohe Marx, 2013) bound bridged the gap
  - tight bound on join size, dependent on fractional edge cover of the hypergraph (more later)
- NPRR (Ngo, Porat, Re, Rudra, 2012) came up with first algorithm - pretty complicated
- Leapfrog Triejoin (2012) was a prexisting algorithm that was shown to be optimal as well

Hardness

- Typical "minimally hard" cyclic query, counting triangles

> Q = R(A, B) ⋈ S (B, C) ⋈ T (A, C)

- Instance:
  - $R = \{a_0\} \times \{b_0, \ldots, b_m\} \cup \{a_0, \ldots, a_m\} \times \{b_0\}$,
  - $S = \{b_0\} \times \{c_0, \ldots, c_m\} \cup \{b_0, \ldots, b_m\} \times \{c_0\}$,
  - $T = \{a_0\} \times \{c_0, \ldots, c_m\} \cup \{a_0, \ldots, a_m\} \times \{c_0\}$
- Each relation has:
  - $N = 2m + 1$ tuples
  - $|Q| = 3m + 1$
  - but each pairwise join has $m^2 + m$
- AGM bound, dependent on fractional edge cover of hypergraph
  - Basically, we need every single variable to be covered by picking edges, possibly fractionally
    - Here picking an edge means picking a relation
    - Assigning x_i to each edge R_i.
  -
    - $|Q| \leq \Pi_i |R_i|^{x_i}$
    - where x_i denotes the fraction by which we are picking R_i



  -
- Here:
  - $|Q| \leq |R|^{0.5}|T|^{0.5}|S|^{0.5}$, essentially $O(N^{3/2})$
  - $|Q| \leq |R||T|$
- Q: when is first bound worse than the second? and vice-versa?
  - When all three relations are same size, former is tighter bound
  - When R and T are single tuples while S is size N, then latter is better

Generic Join vs. Binary Join

- Use slightly different notation to match freejoin paper
  - Our query: $Q\ (x,\ y,\ z)$:- $R(x,\ y)$, $S\ (y,\ z)$,$T\ (z,\ x)$.
- Focus on left-deep "linear" plans

```
for (x, y) in R:
  s = S[y]?
  for (y, z) in s:
    t = T[x,z]?
    for (x, z) in t:
      output(x, y, z)
```

(a) Binary join.

```
for a in R.x ∩ T.x:
  r = R[a]; t = T[a]
  for b in r.y ∩ S.y:
    s = S[b]
    for c in s.z ∩ t.z:
      output(a, b, c)
```

(b) Generic Join.

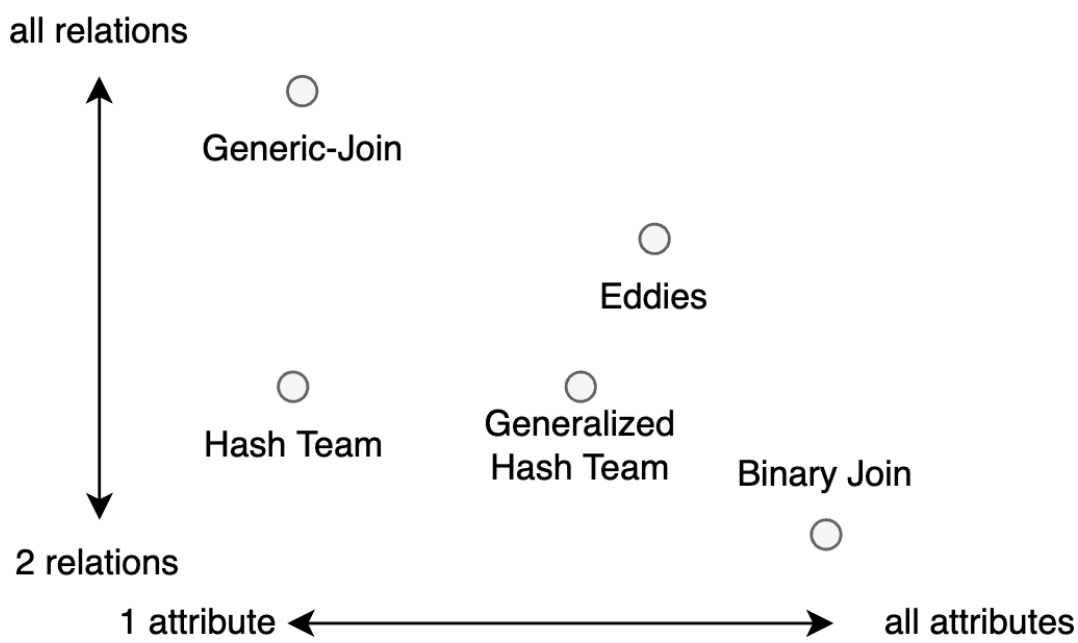
Figure 2: Execution of binary join and Generic Join for $Q_\triangle$. The notation S[y]? performs a lookup on S with the key y, and continues to the enclosing loop if the lookup fails. Binary join iterates over *tuples*, Generic Join iterates over *values*.

- Both joins are entire nested loops over something (tuples in relations vs variables)
- Binary join first
  - $R \bowtie S \bowtie T$
  - Build hash tables on S and T
  - Probe S's HT with R's tuple, probe T's HT with result
- Generic Join - based on Leapfrog Triejoin
  - Drawn from "Skew Strikes Back" paper
  - Loops are over variables, not tuples
  - Pick an order of variables, here, x, then y, then z
  - First compute intersection of all x values across all relations (here R, T)
  - For each such value, a, compute the residual GJ | x = a
    - That is, each relation R is replaced with $\sigma_{x=a}R$
    - That is, for each a in $\cap\{\ \Pi_x(Ri)$ | Ri contains $x\ \}$
      - compute Q(a/x)
- Implementation of GJ:
  - Uses hash-tries, one per relation in the query
    - Depth is equal to one plus the number of attributes,
      - each node is either empty leaf or a hash map mapping each atomic value to another node.
    - For example, for R, order of nodes in the hash trie is x then y
      - The first node = hash map indexed on values of x, and will point to hashmaps at the next node on values of y, which will point to empty leaf node
    - To construct intersection of R's x values and T's x values, we look up both hashmaps and perform the intersection
- GJ's implementation has issues...
  - From a online complexity standpoint matches AGM bound via $|R|^{0.5}|T|^{0.5}|S|^{0.5}$

- But does not include complexity of trie creation
  - For example, if T is significantly larger than R, S, then the run time of Generic Join is $\ll |T|$, yet during preprocessing Generic Join needs to read the entire relation T
- Plus additional downsides because:
  - Hash tries are not as optimized/easy to construct as hash tables
  - No vectorization, columnar execution common in binary joins

Free Join

- Wants to be able to be more nimble/generalize GJ and Binary Join
  - Closer to Eddies than the two extremes
  - Rather than: pick a variable, do intersection of all relations (GJ)



- Introduces a structure called GHT (Generalized Hash Trie),
  - GHT generalizes hash tables and hash tries
  - A GHT is a tree where each leaf is a vector of tuples, and each internal node is a hash map whose keys are tuples, and each key maps to a child node.
  - The schema of a GHT is the list $[\boldsymbol{y}0, \boldsymbol{y}1, \ldots, \boldsymbol{y}\ell]$ where $\boldsymbol{y}k$ are the attribute names of the key at level k.
  - Hash-trie: GHT where each key is a tuple of size 1, leaves are empty vectors
  - Hash-table: two-level GHT where leaves are tuples, level 0 stores keys
- Additionally, the Free Join plan generalizes binary plans and Generic Join plans, and the Free Join algorithm generalizes binary join and Generic Join.

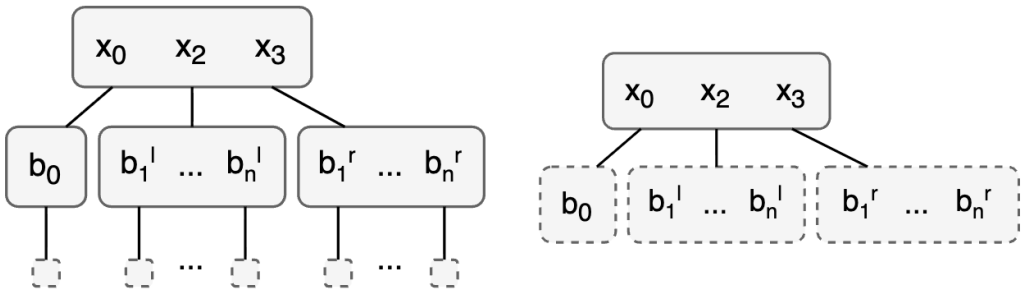


Figure 6: Two GHTs. The one on the left is also a hash trie, and the one on the right is similar to a hash table. Each box with solid border stores hash keys, and each box with dashed border is a vector of tuples. An empty box is an empty vector, representing a leaf.

Free Join: Key Ideas`

- Essentially binary join and GJ are both nested loops varying in granularity:
  - Binary iterates over whole tuples, one relation per loop level (full rows)
  - GJ iterates over one variable at a time, touching all relations (full columns)
- Here, we instead want to, at each level, choose which variables and which relations (basically partial rows/columns)
- Represent via bracket notation - plan = `[ node1, node2, ... ]`
- Each node is a list of **subatoms**: `[ R(x,a), S(x), T(x) ]`
  - A subatom `R(x,a)` means "variables x and a, from relation R."
  - Every variable of every relation must appear in exactly one node; we're essentially distributing columns of each table across loop levels
- Within each node:
  - The **cover** (listed first) is the subatom you *iterate over* — it introduces new variables
  - All other subatoms are *probed* using variables already bound
- For the clover query `Q♣(x,a,b,c) :- R(x,a), S(x,b), T(x,c)`:
  - `[ [R(x,a), S(x), T(x)], [S(b)], [T(c)] ]`
  - **Node 1**: iterate R tuples (x,a); probe S and T on x; skip if either fails
  - **Node 2**: iterate b-values in S(x) retrieved above
  - **Node 3**: iterate c-values in T(x) retrieved above

Key optimization: factoring

- starting from a binary join plan, FJ can pull probes to **earlier loop levels** whenever their key variables are already bound there.
  - filters dead-end tuples before the inner loop ever runs
- e.g., for `Q♣`, the naive binary translation probes `T[x]` *inside* the b-loop
  - wasteful, since x doesn't change there:

```
# Before factoring
for (x, a) in R:
    s = S[x]?
    for b in s:
        t = T[x]? # repeated n times, x never changes
```

```
        for c in t:
        output(x, a, b, c)
```

- after moving `T(x)` to the outer node:

```
# After factoring
for (x, a) in R:
    s = S[x]?
    t = T[x]? # checked once, kills dead-ends early
    for b in s:
    for c in t:
        output(x, a, b, c)
```

- In plan notation:
  `[ [R(x,a), S(x)], [S(b), T(x)], [T(c)] ]` → `[ [R(x,a), S(x), T(x)], [S(b)], [T(c)] ]` ``
- This is strictly between binary and GJ
  - get Generic Join's early filtering without building a trie on R
  - OTOH, FJ is faster than GJ:
    - GJ must build a full trie on *every* relation, including the largest.
    - The left relation in a binary plan is typically large and we would prefer to iterate through it, but GJ will force us to build a trie
      `